Grammar vs Bits:
Format Contracts and Weight Precision in Small-Model Tool Calling

M. Mohith
Chinmaya Vishwa Vidyapeeth, Kerala, India
achumohith14@gmail.com

Abstract

Does strict grammar decoding recover what Q4 quantization loses in small-model tool calling? We find no clear quantization-specific recovery. On the primary JSON-contract point estimates, whole-response constraints helped F16 at least as much as Q4 for both Llama models; only Llama-1B's recovery interval is nominally negative, and Llama-3B's interval includes zero. In a separate sensitivity on the same 60 parallel cases, Q4 whole-strict accuracy was 1.0% (primary JSON) versus 62.3% (Pythonic) for Llama-1B and 0.0% versus 88.3% for Llama-3B. Paired gains of 61.3 [48.7, 73.3] and 88.3 [80.0, 95.7] points far exceed Pythonic F16-to-Q4 gaps of 6.0 and 3.3 points. A format contract means the prompt, parser and grammar. This nonrandomized sensitivity changes all three, separately from the primary BFCL conceptual replication; it does not isolate a parser-only causal effect. Quantization-specific recovery and its change from JSON remain inconclusive in the sensitivity. Intervals are pointwise, unadjusted and specific to this setup. No primary free-decoding precision-loss interval excludes zero. We release frozen inputs, responses, code and paired uncertainty estimates.

Keywords: tool calling, quantization, constrained decoding, grammar-constrained generation, LLM evaluation, small language models

1 Introduction

Does strict grammar decoding recover tool-calling accuracy lost to Q4 quantization? The primary comparison shows no clear quantization-specific recovery. On the primary JSON-contract point estimates, whole-response constraints helped F16 at least as much as Q4 for both Llama models; only Llama-1B's recovery interval is nominally negative, and Llama-3B's interval includes zero. In a separate sensitivity on the same 60 parallel cases, Q4 whole-strict accuracy was 1.0% (primary JSON) versus 62.3% (Pythonic) for Llama-1B and 0.0% versus 88.3% for Llama-3B. The observed format contrasts far exceed Pythonic F16-to-Q4 whole-strict gaps of 6.0 and 3.3 points. This changes prompt, parser and grammar together; it establishes neither a general rule about quantization nor an intrinsic Llama limitation.

TOOLDEC already enforces tool syntax without fine-tuning [17]. Prior work separates structural and content failures and tests grammar mitigation in quantized models. We ask whether strict enforcement changes the *incremental precision gap* differently from permissive or trigger-gated grammars. A quantized model can gain simply because an intervention helps every precision. A grammar admitting arbitrary text cannot guarantee schema-valid output. We therefore use matched full-precision controls and check what each decoder admits.

The benchmark is single-turn prediction; no generated call is executed. The separate sensitivity is not pooled into the primary matrix or interpreted as a parser-only causal experiment [11].

2 Related work

Quantization studies show dependence on recipe and workload [8]. Aggregate agent success can obscure process failures [7]; decision-level analyses already study which tool-use decisions low-bit weights disrupt [15]. These motivate a precision interaction, beyond the general observation that compressed models can make tool errors.

The closest software reports, QuantCall, QuantMCP and Toolhound [2, 3, 4], differ in engine, precision controls, tasks and replication. QuantCall's default grammar unions a schema-shaped call with unrestricted text. QuantMCP describes a related escape and a forced-envelope probe with stalling. Toolhound reports trigger-gated comparisons on MLX and a custom suite. These substantive prior experiments motivate the mechanisms and confirmation criteria in Table 1.

StructureBench and Chavan study small-model structural validity and content accuracy [16, 5]. Repair, Not Improvement isolates stopping and tool-name restrictions in routing, using repeated runs and paired inference [9]. JSONSchemaBench studies schema coverage and efficiency [6]. Constraint Tax shows that final-response grammars can exclude a serving protocol's tool calls [10]. We therefore constrain the declared call envelope and distinguish configuration, initiation, completion and correct-

Table 1: Conceptual replication targets. Agreement concerns mechanisms under changed conditions, not prior numerical scores. Empirical directional support requires a paired interval excluding zero; otherwise that contrast remains inconclusive.

Prior work	What changes here	What would count as confirmation
QuantCall [2]	Shared BFCL subset, prompt/compiler, paired operational schedules; add strict enforcement.	A permissive escape gives little benefit while stricter enforcement changes validity or task accuracy. A null strict effect remains a valid outcome.
QuantMCP [3]	Replace live MCP execution with single-turn BFCL AST scoring; preserve abstention and log completion failures.	An escape can preserve malformed output; strict generation can still fail to terminate or answer correctly. No claim to reproduce MCP task-success rates.
Toolhound [4]	Replace MLX/custom tasks with llama.cpp/BFCL and add matched F16 controls.	Constraints improve call structure more consistently than task correctness. The added precision interaction need not match a q4-only gain.

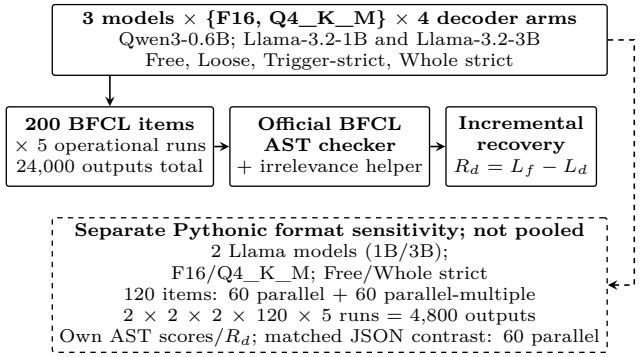


Figure 1: Primary matrix (solid) and separate format sensitivity (dashed). Primary accuracy uses 180 required-call items; 20 irrelevance cases are scored separately.

ness. Serving-stack differences can independently change apparent tool fidelity [14]. Our conceptual replication retests mechanisms without reproducing another engine or live MCP execution.

3 Experimental design

3.1 Frozen tasks, models and precision

We evaluate `Qwen3-0.6B`, `Llama-3.2-1B-Instruct`, `Llama-3.2-3B-Instruct`. The frozen sample contains 200 original cases: 60 simple, 60 multiple, 60 parallel, 20 irrelevance. The primary required-call subset contains 180 items. Each precision–decoder cell has 5 operational runs, totaling 24,000 benchmark generations in the complete matrix. Decoding uses temperature 0.0, 4 parallel server slots, a 4096-token input-plus-output budget per request, and an output cap of 512 tokens. All conditions share original items and accepted answers, selected without model predictions. The pilot calibrates runtime and cost. Prompts are frozen from pinned publisher templates with a fixed Llama template date.

Weights are exported once to F16 GGUF and the quantized artifact is generated directly from that export. The checkpoint's storage dtype may differ from the F16 reference. Both K and V caches stay F16. Qwen thinking is disabled [13]. Model revisions, source and output hashes, conversion commands, server arguments and the pinned llama.cpp source [1] are recorded. Each model uses one A10 worker for all arms and precisions, with a fixed Modal allocation selector. Records retain device-reported A10/A10G labels and drivers; model sessions may use different physical hosts.

3.2 Decoder languages and prompt contract

Each model's arms share rendered prompts and offered tools. The primary Llama prompts use the single-object JSON tool branch of `tokenizer_config.json`'s embedded HF chat template, with `tools_in_user_message=True`. Both revisions share the template hash in Appendix A. We added a system-message JSON-array convention for parallel calls, but the HF template's later user-message instruction requests one JSON object. These instructions conflict on parallel tasks. Meta documents Pythonic call lists [11], which the primary JSON contract does not evaluate. Llama findings concern this declared contract, not its best supported interface. Qwen uses tagged call objects with `name` and `arguments`; Llama uses objects with `name` and `parameters`, or arrays for parallel calls. All arms use `NO_CALL` when no offered tool applies. The grammar never uses the expected function, accepted values or gold call count.

Free applies no grammar. **Loose** unions the schema-shaped language with arbitrary text, reproducing the escape as a control. **Trigger-strict** leaves initiation unconstrained and activates the call language at the declared prefix. Qwen uses its preserved call token; Llama uses one pattern matching the earliest object or array opener, including an opener merged into a token. **Whole strict** admits only a documented call sequence or the explicit no-call form. Parallel-call count and order are not fixed to the reference. Lazy activation can encounter a merged trigger-containing token whose remainder violates the call language. Known replay errors are retained as failed attempts, rather than retried or admitted through a grammar escape.

The compiler allows documented object keys in any order, requires specified keys and forbids duplicates. As an explicit API policy, known property sets are closed; free-form dictionaries stay free-form. For nested BFCL

dictionaries with untyped required fields, names are retained but values are unconstrained; answers never supply types. These normalization choices are released as part of the intervention.

Finite character-level membership checks test selected schemas, malformed strings, unknown functions, missing or extra arguments, types, key order, parallel calls and abstention. These show tested coverage, without proving every JSON Schema feature or token-level behavior. Before collection, synthetic runtime probes force a known trigger to verify literal replay and the server's configured lazy mode. These are setup checks, not benchmark successes. No engine patch is used.

3.3 Execution and scoring

Decoding uses temperature zero and shuffled requests, with seed and schedule labels frozen before execution. Identical greedy outputs remain operational repeats, not independent items. Context and output limits, slots, cache settings and stop strings match across arms. Grammar acceptance itself changes when termination is legal. Llama uses publisher token IDs to avoid duplicating the template's beginning-of-text marker. An earlier collection with that duplication remains an aborted setup attempt, stopped before accuracy inspection. Qwen prompts remain strings, using publisher settings that disable an added beginning-of-text token. Logs retain cache-reuse timings and actual generation settings. The cloud supervisor checkpoints events and enforces queue, runtime and scheduling limits frozen before execution. The researcher permitted concurrent full studies after separate pilots; these runs do not establish isolated cross-model throughput.

Official BFCL AST matching determines correctness [12]. We parse the declared JSON envelope into calls without executing them. The pinned checker is unmodified; a minimal registry entry supplies its name-normalization flag without importing unrelated API clients. The unchanged official irrelevance helper uses the same parsing adapter. Irrelevance can credit outputs with no parsed call even when our format diagnostic rejects their prose; we report these separately. Required-call accuracy is the primary endpoint.

Our deterministic diagnostics retain multiple flags and an ordered primary label: format failure, wrong or hallucinated function, missing/extra or hallucinated arguments, wrong type and wrong value, plus missing/extra calls and operational statuses. A schema-valid parallel mismatch is not labeled a known value error without evidence from the checker. Incomplete output and runtime failures remain visible. Unstarted budget-censored cells are reported as incomplete, rather than filled with synthetic predictions.

3.4 Estimands and paired uncertainty

Let $A_{p,d}$ be official item accuracy at precision p and decoder d, with f denoting free decoding. For the quantized precision q, define

$$L_f = A_{\mathrm{F16},f} - A_{q,f}, \tag{1}$$

$$L_d = A_{\mathrm{F16},d} - A_{q,d}, \tag{2}$$

$$R_d = L_f - L_d. \tag{3}$$

R_d removes the decoder effect already present at F16. We also compare $R_{\mathrm{whole}} - R_{\mathrm{loose}}$ and the corresponding trigger contrast on paired observations. These are intervention effects, not an identified fraction of purely syntactic knowledge loss: schemas restrict names, keys and types, and can redirect subsequent value choices.

Primary percentile intervals use crossed paired resampling of original item IDs and operational-run labels, sharing draws across all conditions and models. Item-only intervals provide a sensitivity check. Category rates use their original item subsets. Intervals are pointwise and unadjusted for multiple comparisons; exploratory contrasts are labeled. Without observed failures, empirical bootstrap intervals can degenerate and cannot bound rare unobserved population failures. The recovery share R_d/L_f is undefined for nonpositive observed free loss. We withhold its interval when bootstrap denominators are unstable and never clip values into a partition.

4 Results

For Qwen3-0.6B, Q4 required-call accuracy rises from 70.7% with free decoding to 75.6% with whole-response constraints. Its incremental recovery is 2.1 [-0.7, 5.2] percentage points. Across all cases, Q4 whole-strict response-structure validity is 100.0 [100.0, 100.0]%, while official accuracy is 73.0 [66.9, 79.0]%.

For Llama-3.2-1B, Q4 required-call accuracy rises from 14.2% with free decoding to 52.7% with whole-response constraints. Its incremental recovery is -4.7 [-9.1, -0.3] percentage points. Across all cases, Q4 whole-strict response-structure validity is 100.0 [100.0, 100.0]%, while official accuracy is 47.4 [40.5, 54.2]%. Its Q4 whole-strict parallel accuracy under the declared JSON contract is 1.0 [0.0, 4.0]%. This does not measure the publisher Pythonic call-list contract.

For Llama-3.2-3B, Q4 required-call accuracy rises from 29.2% with free decoding to 61.0% with whole-response constraints. Its incremental recovery is -2.1 [-5.0, 0.6] percentage points. Across all cases, Q4 whole-strict response-structure validity is 100.0 [100.0, 100.0]%, while official accuracy is 54.9 [48.0, 61.7]%. Its Q4 whole-strict parallel accuracy under the declared JSON contract is 0.0 [0.0, 0.0]%. This does not measure the publisher Pythonic call-list contract.

23,860 generations completed normally, 139 reached the 512-token cap, and one failed during grammar replay.

Table 2: Required-call accuracy (%) and crossed paired 95% intervals. Repeated schedules remain clustered with original items.

Model	Precision	Free	Loose	Triggered	Whole strict
Qwen3-0.6B	F16	73.9 [67.2, 80.0]	73.9 [67.2, 80.0]	76.7 [70.0, 82.8]	76.7 [70.0, 82.8]
Qwen3-0.6B	Q4_K_M	70.7 [64.0, 77.0]	70.7 [63.9, 77.0]	74.8 [68.4, 80.7]	75.6 [69.2, 81.4]
Llama-3.2-1B	F16	12.8 [8.3, 17.8]	12.8 [8.3, 17.8]	54.3 [47.1, 61.7]	55.9 [48.7, 63.1]
Llama-3.2-1B	Q4_K_M	14.2 [9.3, 19.4]	14.1 [9.2, 19.4]	51.9 [44.7, 59.1]	52.7 [45.3, 59.9]
Llama-3.2-3B	F16	27.2 [20.6, 33.9]	27.2 [20.6, 33.9]	61.1 [53.9, 68.3]	61.1 [53.9, 68.3]
Llama-3.2-3B	Q4_K_M	29.2 [22.7, 35.9]	29.4 [22.8, 36.1]	60.4 [53.3, 67.6]	61.0 [53.9, 68.1]

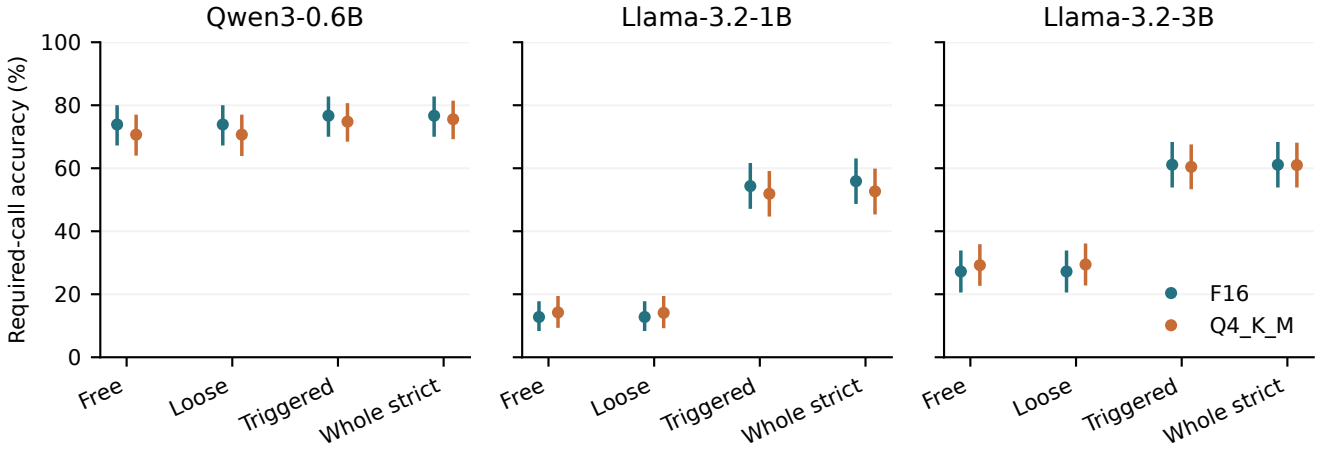


Figure 2: Required-call accuracy under each intervention. Whiskers are crossed paired bootstrap intervals, not independent-binomial intervals over repeated generations.

Negative incremental recovery does not mean constraints lowered Q4 accuracy: they helped F16 more in these cells. Valid structure includes no-call responses; task correctness and irrelevance are scored separately. Other diagnostic rates, category endpoints, item-only sensitivity intervals, paired decoder contrasts and raw responses are released with the result tables. Intervals crossing zero do not establish equivalence.

Separate Pythonic-format sensitivity. We collected 4,800 responses on 120 cases: the original parallel and a separately sampled parallel-multiple subset. Both Llama models used both precisions, free and whole-strict decoding, and the same operational labels. The prompt uses Meta's call-list syntax, omits the HF tools instruction and uses `[]` for abstention. Llama-3.2-1B Q4 whole-strict parallel accuracy is 62.3 [50.0, 74.3]%, versus 1.0% under the original JSON contract. Its Pythonic parallel incremental recovery is 1.7 [-10.3, 13.7] points; the paired change in recovery versus JSON is 2.3 [-10.0, 15.0] points. Llama-3.2-3B Q4 whole-strict parallel accuracy is 88.3 [80.0, 95.7]%, versus 0.0% under the original JSON contract. Its Pythonic parallel incremental recovery is -3.3 [-8.7, 0.7] points; the paired change in recovery versus JSON is -3.3 [-8.7, 0.7] points. The paired Q4 contract gains are positive. Sensitivity parallel incremental recovery and its protocol change remain inconclusive. Only the original parallel items are matched across the changed prompt, parser and grammar. Parallel-multiple scores are separate. Sessions differ in time and host; this is not a randomized crossover. A platform interruption left 1,610 saved Llama-3B outputs. Another A10 collected the remaining 790 responses, retaining all saved failures. The primary matrix is unchanged.

Sensitivity token limits. The 578 token-limit outcomes fall as follows; each cell has 600 observations.

	F16		Q4_K_M	
Llama	Free	Whole strict	Free	Whole strict
1B	146	65	284	64
3B	5	5	4	5

All limits already count as incorrect in the reported accuracies and intervals. The large failure-inclusive contract gains therefore remain. Sensitivity parallel incremental recovery and its protocol change remain inconclusive. Conclusions concern the fixed output cap, not unlimited generation. No added exclusion analysis is claimed.

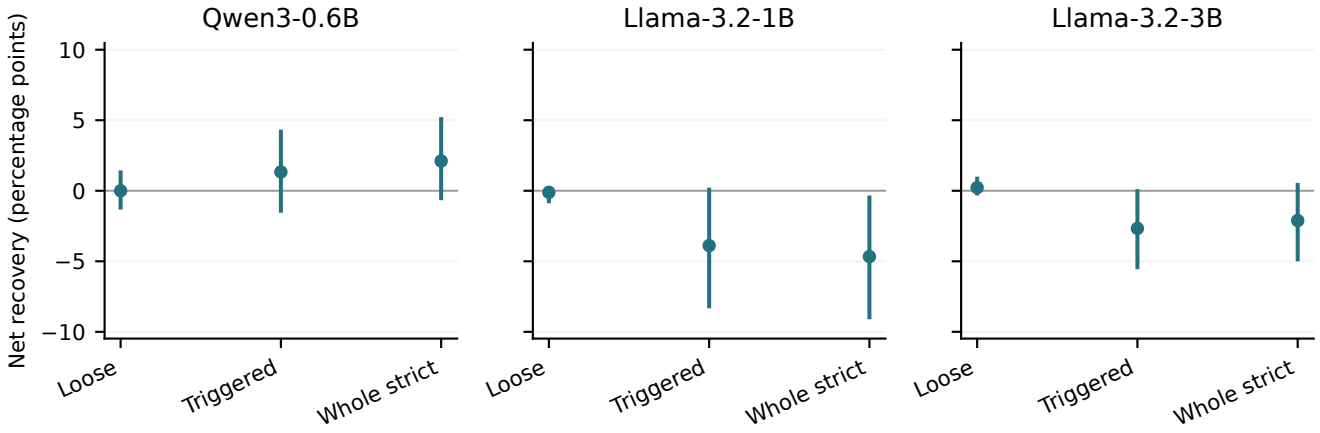


Figure 3: Signed recovery of the incremental F16–Q4 gap. Zero means no observed reduction of that gap; negative effects are retained. This quantity does not causally separate syntax from semantics.

Table 3: Required-call signed precision loss and incremental recovery, in percentage points [95% CI]. The final column is the normalized share (%) when its denominator is stable.

Model	Decoder	Free loss	Constrained loss	Net recovery	Recovery share
Qwen3-0.6B	Loose	3.2 [-1.9, 8.3]	3.2 [-1.9, 8.3]	0.0 [-1.3, 1.4]	unstable
Qwen3-0.6B	Triggered	3.2 [-1.9, 8.3]	1.9 [-2.4, 6.2]	1.3 [-1.6, 4.3]	unstable
Qwen3-0.6B	Whole strict	3.2 [-1.9, 8.3]	1.1 [-3.2, 5.3]	2.1 [-0.7, 5.2]	unstable
Llama-3.2-1B	Loose	-1.4 [-4.4, 1.4]	-1.3 [-4.4, 1.6]	-0.1 [-0.9, 0.3]	undefined
Llama-3.2-1B	Triggered	-1.4 [-4.4, 1.4]	2.4 [-1.8, 6.7]	-3.9 [-8.3, 0.2]	undefined
Llama-3.2-1B	Whole strict	-1.4 [-4.4, 1.4]	3.2 [-0.7, 7.2]	-4.7 [-9.1, -0.3]	undefined
Llama-3.2-3B	Loose	-2.0 [-4.6, 0.3]	-2.2 [-4.9, 0.2]	0.2 [-0.3, 1.0]	undefined
Llama-3.2-3B	Triggered	-2.0 [-4.6, 0.3]	0.7 [-0.8, 2.2]	-2.7 [-5.6, 0.1]	undefined
Llama-3.2-3B	Whole strict	-2.0 [-4.6, 0.3]	0.1 [-1.3, 1.4]	-2.1 [-5.0, 0.6]	undefined

5 Cost, limitations and practical implications

The pilot-based cost forecast for the primary campaign was $4.74–$8.14. Before the follow-up, reported gross project spending was $2.6110, including setup and interrupted attempts, against the $12.00 cap. These are reported charges, not a settled invoice. Final app and container shutdown records are stored separately from the billing snapshot. The separate Pythonic sensitivity added $0.7346, including failed and interrupted apps, bringing reported project spending to $3.3456, within the additional $3.00 and total $12.00 caps. Forecasts are assumptions. Timings omit unmeasured platform stages; metering can lag and cannot establish storage completeness. Earlier failures remain released.

The frozen BFCL sample is not a complete leaderboard evaluation. AST correctness establishes neither safe execution, successful multi-turn agents nor correctness on unseen APIs. The irrelevance sample is small. Fixed inference, cache dtype, weight recipe, envelope and schema normalization limit generalization to other quantizers, KV-cache compression, larger catalogs or engines. Prefix triggering is an operational rule, not a direct measure of internal intent; log-derived activation is not a direct sampler-state trace.

Primary failures and post-hoc exclusion. Qwen3-0.6B has 0 token-limit outcomes and 0 replay errors; Llama-3.2-1B has 101 token-limit outcomes and 1 replay error; Llama-3.2-3B has 38 token-limit outcomes and 0 replay errors. Failures cluster by decoder and task; exclusion may depend on treatment or difficulty. The analysis frozen before execution counts failures as incorrect. Three sensitivity checks exclude failures per cell, remove a model–item–run from all precision–decoder cells if any cell failed, or remove whole items with any failure. The second rule preserves a common observed cohort; its required-call whole-strict recovery is Qwen3-0.6B 2.1 [-0.7, 5.2]; Llama-3.2-1B -4.8 [-9.4, -0.5]; Llama-3.2-3B -2.1 [-5.1, 0.6] percentage points [95% CI]. Across all three rules and both endpoints, directional interval classifications change for 0 recovery effects and 0 decoder contrasts. Cell-only exclusion changes the completed population; all three select on a post-treatment outcome and may favor easier items. None estimates performance with unlimited generation or a repaired engine. The failure report gives every cell count, retained denominator and supplementary interval; primary results remain unchanged.

Closed schemas can forbid useful undocumented arguments; typed values can still be wrong. Parser policy affects prose, abstention and parallel-call scoring. Operational repeats cover neither new training runs, quantization builds or templates nor generalization across hardware. Intervals crossing zero do not prove equality.

Operators need task accuracy under the same serving contract and a matched high-precision control. Structural guarantees remain useful without accuracy gains. The

planned follow-up compares schema-audit and precision-fallback methods with ordinary retries; these methods are not evaluated here.

6 Conclusion

The separate sensitivity shows that the tool-call format contract has a much larger observed accuracy contrast than weight precision in this setup. Llama-1B Q4 whole-strict parallel accuracy rises from 1.0% to 62.3%; Llama-3B Q4 whole-strict parallel accuracy rises from 0.0% to 88.3%. This changes the interpretation of the low primary Llama parallel scores, while leaving the primary matrix unchanged. The comparison changes prompt, parser and grammar together. Sensitivity parallel incremental recovery and its protocol change remain inconclusive. The pointwise, unadjusted intervals do not establish recovered quantization damage. No primary free-decoding precision-loss interval excludes zero.

A Settings and availability

Code, frozen data, responses, analysis and the paper are at `https://github.com/mohi-devhub/grammar-vs-bits`. The primary campaign is `full-002`; the Pythonic sensitivity is separate. The design was frozen before execution, without a public preregistration timestamp. The code is MIT licensed; redistributed BFCL material includes Gorilla's Apache-2.0 license and attribution.

BFCL v4 data and checker are pinned to Gorilla revision `6ea57973c7a6097fd7c5915698c54c17c5b1b6c8`. The engine is llama.cpp revision `8542e94350f0ade01644db6f1d1520da3814dafd`. Crossed bootstrap intervals use 10,000 draws and seed 20261028. Operational labels are 101, 211, 307, 401 and 503. Qwen runs in non-thinking mode (`enable_thinking=False`). Each model loads F16 first and Q4 second, with the five shuffled schedules inside each precision block; precision order was not counterbalanced. These intervals describe variation across items and schedules under that execution.

The Llama HF JSON templates are from 1B revision `9213176726f574b556790deb65791e0c5aa438b6` and 3B revision `0cb88a4f764b7a12671c53f0838cd831a0843b95`. Their template SHA-256 is `2732d9da5568668ed22ea202bffb12f0df405439651e1275ba1f521522f92966`. `provenance/` holds exact executed worker sources and hashes, separately from maintained code. Historical launcher revisions were not retained; no recovery is claimed. Prompts and per-run commands are released.

References

[1] llama.cpp. Pinned software. `https://github.com/ggml-org/llama.cpp/tree/8542e94350f0ade01644db6f1d1520da3814dafd`.

[2] Quantcall. Pinned software and experiment reports. `https://github.com/Happynood/quant-toolcall-bench/tree/6b6e29e5c83a91a52bdfc68b5f3063de172c0e55`.

[3] Quantmcp. Pinned software and experiment reports. `https://github.com/Happynood/quant-mcp-bench/tree/fc98a47c6a0ef77bb8be3a915016b42d203faf45`.

[4] Toolhound. Pinned software and experiment reports. `https://github.com/Code-byte404/toolhound/tree/97666be79a2af2afb78b36a168a5f89f84be6d97`.

[5] Akash Chavan. Constrained decoding eliminates structural failures in small llms but reveals a scale-dependent semantic gap. arXiv:2609.23742v1, 2026. `https://arxiv.org/abs/2609.23742v1`.

[6] Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. Jsonschemabench: A rigorous benchmark of structured outputs for language models. arXiv:2501.10868v3, 2025. `https://arxiv.org/abs/2501.10868v3`.

[7] Jiwon Jang, Kisu Yang, Heuiseok Lim, and Hyunwoo Park. Flat score, amplified failures: How the error budget masks damage in quantized llm agents. arXiv:2607.27275v1, 2026. `https://arxiv.org/abs/2607.27275v1`.

[8] Eldar Kurtic, Alexandre Marques, Shubhra Pandit, Mark Kurtz, and Dan Alistarh. "Give Me BF16 or Give Me Death"? Accuracy-Performance Trade-Offs in LLM Quantization. arXiv:2411.02355v4, 2024. `https://arxiv.org/abs/2411.02355v4`.

[9] Janghoon Lee. Repair, not improvement: Decomposing constrained decoding in tool-call abstention. arXiv:2608.13959v2, 2026. `https://arxiv.org/abs/2608.13959v2`.

[10] Fangzheng Li, Aimin Zhang, and Chen Lv. Constraint tax in open-weight llms: An empirical study of tool calling suppression under structured output constraints. arXiv:2606.25605v1, 2026. `https://arxiv.org/abs/2606.25605v1`.

[11] Meta. Llama 3.2 text prompt format. `https://github.com/meta-llama/llama-models/blob/0e0b8c519242d5833d8c11bffc1232b77ad7f301/models/llama3_2/text_prompt_format.md`.

[12] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. Proceedings of Machine Learning Research, volume 267, 2025. `https://proceedings.mlr.press/v267/patil25a.html`.

[13] Qwen Team. Qwen3-0.6B model card, 2025. `https://huggingface.co/Qwen/Qwen3-0.6B/tree/c1899de289a04d12100db370d81485cdf75e47ca`.

[14] Lijuan Tang and Yuemeng Zheng. Measuring the serving stack instead of the model: Hidden confounds in local tool-use evaluation. arXiv:2609.26693v1, 2026. `https://arxiv.org/abs/2609.26693v1`.

[15] Zekun Wu, Swati Dhiman, and Adriano Koshiyama. Which decisions low-bit quantization breaks, and how

to predict them. arXiv:2608.06564v5, 2026. https://arxiv.org/abs/2608.06564v5.

[16] Xiaokun Xiong, Zhengjie Xu, Junyi Chen, Shihao Bai, Ruihao Gong, and Xianglong Liu. Structurebench: A unified benchmark suite for multi-scenario structured generation tasks with on-device models. IJCAI proceedings, 2026. https://ijcai.org/proceedings/2026/267.

[17] Kexun Zhang, Hongqiao Chen, Lei Li, and William Wang. Don't fine-tune, decode: Syntax error-free tool use via constrained decoding. arXiv:2310.07075v3, 2024. https://arxiv.org/abs/2310.07075v3.